

Distributed Training

Project 3 — Single GPU → DDP → FSDP → Ray Train

A-Tier Skill · 28/41 JDs · Llama-3-8B · 4× A10G · Updated May 12, 2026

The pitch: Fine-tuned Llama-3-8B across 4× A10G GPUs — benchmarked single-GPU baseline vs PyTorch DDP vs FSDP vs Ray Train. Measured samples/sec, scaling efficiency, and GPU memory per rank at each stage. Key finding: 8B model requires gradient checkpointing under DDP (16GB/rank); FSDP drops memory to ~4GB/rank — no checkpointing needed, larger batch, better GPU utilization. Ray Train eliminates manual process group setup at minimal throughput cost.

The Story This Project Tells

Problem	Llama-3-8B in FP16 = 16GB. A single A10G (24GB) can fit it but leaves almost no room for activations or optimizer state
Solution	FSDP shards model weights, gradients, and optimizer states across GPUs. Each of 4 GPUs holds only ~4GB of the model
Result	DDP: 16GB/rank, gradient checkpointing required, small batch. FSDP: ~4GB/rank, no checkpointing needed, 4x larger batch
Why it matters	28/41 JDs mention distributed training. The DDP vs FSDP memory story on a real 8B model is exactly what senior ML

Model + GPU Setup

Config	GPU	Count	Cost/hr	Why
All stages	A10G (24GB)	4×	~\$1.40/hr	8B FP16 = 16GB — A10G is tight for DDP, comfortable for FSDP
Single GPU baseline	A10G (24GB)	1×	~\$0.35/hr	8B + grad checkpointing fits; measures baseline samples/sec

Model: Llama-3-8B — 16GB in FP16. This is the right size: DDP fits but requires gradient checkpointing (forces small batch), FSDP drops to ~4GB/rank (no checkpointing, larger batch — dramatic, explainable difference). Fine-tune on AG News (4-class text classification) or Alpaca (instruction tuning). **GPU:** 4× A10G on Vast.ai (~\$1.40/hr). Full session ~6 hrs = ~\$9. **Connect via VS Code Remote-SSH** — same workflow as inference project.

What to Measure — The Numbers That Matter

Metric	How to measure	Why it matters
samples/sec	count samples processed per second during training	Primary throughput metric — higher = training finishes faster
scaling efficiency	$(N\text{-GPU samples/sec}) / (N \times 1\text{-GPU samples/sec}) \times 100\%$	Shows how well distributed training scales — 100% = perfect linear, real world 70-90%
GPU memory / rank	<code>nvidia-smi --query-gpu=memory.used</code> per process	DDP: full model on every GPU. FSDP: model split across GPUs — memory drops
time to loss X	wall clock time to reach target validation loss	The real business metric — how long does a training run actually take
communication overhead	$(\text{ideal linear throughput} - \text{actual throughput}) / \text{ideal linear throughput}$	Quantifies all-reduce cost — grows with model size and number of GPUs

Build Stages — 4 Stages, One Question Each

Stage 1 Single GPU Baseline — Llama-3-8B on 1× A10G

- Fine-tune Llama-3-8B on AG News (4-class) — standard PyTorch loop
- 8B FP16 = 16GB weights. A10G has 24GB — only 8GB left for activations
- Enable gradient checkpointing immediately — without it, you OOM on first backward
- Log to MLflow: loss, samples/sec, GPU memory (nvidia-smi), batch size
- Run for 200 steps — record wall clock time, samples/sec, peak GPU memory
- This is the 'before' — constrained batch size, slow, one GPU doing all the work

Key files:

- train_single.py — training loop with gradient checkpointing + MLflow
- utils/metrics.py — samples/sec and GPU memory helpers

Stage 2 PyTorch DDP — Data Parallel on 4× A10G

- Wrap model: model = DDP(model, device_ids=[local_rank])
- Launch: torchrun --nproc_per_node=4 train_ddp.py
- DDP replicates full 16GB model on every GPU — still needs gradient checkpointing
- Each GPU processes a different data shard — gradients all-reduced after backward
- Run at 1, 2, 4 GPUs — record samples/sec, scaling efficiency, memory/rank
- nvidia-smi: all 4 GPUs show ~16–20GB used — full model on every rank
- Scaling efficiency target: 80–88% at 4 GPUs (all-reduce cost on 16GB model)

Key files:

- train_ddp.py — DDP training with torchrun, rank-aware checkpoint saving
- launch_ddp.sh — torchrun command with all flags
- checkpointing.py — save on rank 0 only, resume from any rank

Stage 3 PyTorch FSDP — Sharded Model, 4× A10G

- Wrap model: `model = FSDP(model, auto_wrap_policy=transformer_auto_wrap_policy)`
- FSDP shards weights, gradients, AND optimizer states — each GPU holds ~4GB
- No gradient checkpointing needed — 20GB freed for activations and larger batch
- Run same benchmark — compare samples/sec vs DDP
- `nvidia-smi`: memory/rank drops from ~18GB (DDP) to ~6GB (FSDP) — show this
- Increase batch size until FSDP OOMs — compare max batch vs DDP max batch
- The key interview answer: FSDP doesn't just save memory, it enables larger batches

Key files:

- `train_fsd.py` — FSDP with `transformer_auto_wrap_policy` + mixed precision
- `fsdp_config.py` — `ShardingStrategy`, `MixedPrecision`, `BackwardPrefetch` config

Stage 4 Ray Train — Same DDP, Different Runtime

- `pip install ray[train]`
- Convert `train_ddp.py` to Ray Train: wrap in `train_func`, use `ray.train.torch`
- `ray.init()` — auto-detects all 4 GPUs, no manual `dist.init_process_group`
- Run same benchmark as DDP — compare throughput and setup complexity
- Count lines of setup code: `torchrun` DDP vs Ray Train — Ray wins on simplicity
- Add Ray Tune: 10 lines to turn training into a hyperparameter sweep
- Record: throughput vs DDP, setup lines of code, time to first run

Key files:

- `train_ray.py` — Ray Train version of DDP (same model, different launcher)
- `ray_tune.py` — hyperparameter sweep with Ray Tune over LR and batch size

Benchmark Results Table — Fill In Real Numbers

Run every stage, record every cell. The scaling efficiency column is what interviewers ask about.

Backend	GPUs	samples/sec	Scaling eff.	Mem/rank	Notes
Single GPU (baseline)	1	—	100%	~18–20GB	Grad checkpointing required
DDP	2	—	—%	~18–20GB	Still needs grad checkpointing
DDP	4	—	—%	~18–20GB	Target: 80–88% efficiency
FSDP	4	—	—%	~4–6GB	No checkpointing, larger batch
Ray Train DDP	4	—	~DDP%	~18–20GB	Simpler setup vs torchrun

Day-by-Day Build Plan

Day 1 Setup + Code (no GPU needed)

- Create new repo: distributed-training
- Write train_single.py: Llama-3-8B fine-tune on AG News, gradient checkpointing, MLflow
- Write utils/metrics.py: samples/sec and GPU memory measurement
- Write train_ddp.py: DDP wrapper, torchrun launcher, rank-aware checkpointing
- Write train_fsdp.py: FSDP with transformer_auto_wrap_policy, mixed precision
- Write train_ray.py: Ray Train version of DDP
- Validate data pipeline locally on CPU (tiny model) — no GPU cost
- Push to GitHub

Day 2 Rent GPU — Single GPU + DDP Numbers

- Rent 4× A10G on Vast.ai (~\$1.40/hr) — connect VS Code via Remote-SSH
- Run train_single.py on 1 GPU — hit OOM first without checkpointing (expected)
- Enable gradient checkpointing — record samples/sec, peak memory, batch size
- Run train_ddp.py at 2 GPUs — record numbers
- Run train_ddp.py at 4 GPUs — record numbers, calculate scaling efficiency
- nvidia-smi on all 4 ranks: show ~18–20GB each — DDP replicates full model
- Commit results/single_gpu.json and results/ddp_results.json

Day 3 FSDP + Ray Train + Memory Story

- Run `train_fsd.py` on 4 GPUs — watch memory drop to ~4–6GB/rank
- Remove gradient checkpointing in FSDP — it fits without it
- Increase batch size until FSDP OOMs — compare to DDP max batch size
- Record samples/sec improvement vs DDP (larger batch = better GPU utilization)
- Run `train_ray.py` — compare throughput and count setup lines vs `torchrun`
- Commit `results/fsdp_results.json` and `results/ray_results.json`

Day 4 Ray Train + README + Polish

- `pip install ray[train]`, convert `train_ddp.py` to Ray Train
- Run same benchmark — compare setup complexity and throughput vs `torchrun`
- Fill in all cells in benchmark table
- Write README: story, table, 'why each stage', scaling efficiency section
- MLflow: log all runs, show experiment comparison screenshot in README
- Push final version — project done

Measuring samples/sec — Add This to Every Training Script

```
# utils/metrics.py
import time, torch

class ThroughputTracker:
    def __init__(self):
        self.start = time.perf_counter()
        self.samples = 0

    def update(self, batch_size):
        self.samples += batch_size

    def samples_per_sec(self):
        elapsed = time.perf_counter() - self.start
        return self.samples / elapsed

# In your training loop:
tracker = ThroughputTracker()
for batch in dataloader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    tracker.update(batch['input_ids'].shape[0])

    if step % 10 == 0:
        mlflow.log_metric('samples_per_sec', tracker.samples_per_sec(), step=step)
        mlflow.log_metric('gpu_memory_mb',
            torch.cuda.memory_allocated() / 1024**2, step=step)
```

Scaling efficiency formula: $\text{efficiency} = (\text{N_gpu_throughput} / (\text{N} \times \text{single_gpu_throughput})) \times 100\%$ Example: single GPU = 120 samples/sec. 4-GPU DDP = 420 samples/sec. Efficiency = $(420 / (4 \times 120)) \times 100\% = 87.5\%$. This is the number that goes in your README and on your resume.

Interview Questions — Know These Cold

Q: What is all-reduce and how does DDP use it?

All-reduce is a collective operation where every GPU sends its gradient tensor and receives the sum (or average) across all GPUs. In DDP, after every backward pass, each GPU has computed gradients for its local data shard. All-reduce averages these gradients across all GPUs so every GPU applies the same update. This is what makes DDP correct — every GPU's model weights stay in sync. The cost: you're moving ($\text{model_size} \times 2$) bytes across the network per step. For a 3B model in FP16, that's ~6GB per step per GPU.

Q: What's the difference between DDP and FSDP?

DDP replicates the full model on every GPU. Each GPU has a complete copy. Gradient all-reduce happens after backward. Memory per GPU = full model size. For Llama-3-8B: 16GB of weights on every rank — leaves little room for activations, forces gradient checkpointing, limits batch size. FSDP shards model weights, gradients, AND optimizer states across GPUs. Each GPU holds only $1/N$ of each. Before a forward pass, FSDP all-gathers the full layer weights, computes, then discards them. For Llama-3-8B on 4 GPUs: ~4GB/rank — gradient checkpointing not needed, batch size can be 4x larger, GPU utilization improves. Use DDP when the model fits comfortably on one GPU. Use FSDP when it doesn't.

Q: Why isn't scaling efficiency 100% in DDP?

All-reduce communication overhead. Every step, gradients must be synchronized across all GPUs — this takes time proportional to the model size. With 4 GPUs, you're doing 4x the data throughput but also paying 4x the communication cost. Additionally: data loading can be a bottleneck if not parallelized, and NCCL (the GPU communication library) has setup overhead. In practice, 85-90% efficiency on 4 GPUs is good. Below 70% means something else is the bottleneck — usually data loading or CPU preprocessing.

Q: When would you use Ray Train over torchrun?

torchrun is tightly coupled to PyTorch — it sets up process groups, manages rank/world_size environment variables, and expects you to handle fault tolerance. Ray Train is framework-agnostic and handles worker coordination, fault tolerance, and checkpoint management automatically. The main advantages of Ray: 1) Much simpler multi-node setup — no manual SSH or MPI configuration. 2) Native integration with Ray Tune for hyperparameter search. 3) Built-in fault tolerance — dead workers are automatically restarted. Downside: slight overhead (~5%) and one more dependency. For single-node experiments, torchrun is simpler. For multi-node or when you need HPO, Ray wins.

Q: What is gradient checkpointing and when do you use it?

During a forward pass, PyTorch saves intermediate activations (layer outputs) so the backward pass can compute gradients. For a 3B model with long sequences, these activations can use more memory than the model weights themselves. Gradient checkpointing discards activations after the forward pass and recomputes them during the backward pass. Memory cost: $\sim O(\sqrt{\text{layers}})$. Compute cost: ~30% more compute (you run the forward pass twice). Use it when you're memory-constrained but have compute to spare — typically when increasing batch size would OOM otherwise.

Q: Why does scaling efficiency drop as you add more GPUs?

Two reasons. First, communication cost grows with GPU count — all-reduce with N GPUs requires more network traffic than with 2 GPUs. Second, if your batch is fixed size, each GPU gets a smaller shard — smaller batches are less GPU-efficient (more kernel launch overhead per sample). The right approach: scale batch size with GPU count (global batch = local batch $\times$ N). With a properly scaled batch, efficiency stays high — 85-90% is realistic. Companies with large clusters often use gradient accumulation to simulate larger batches when per-GPU batch size is limited by memory.

Resume Bullets — Fill In Real Numbers

◆ Fine-tuned Llama-3-8B (16GB FP16) across 4× A10G with PyTorch DDP — [X]% scaling efficiency at 4 GPUs; gradient checkpointing required under DDP to fit 8B model alongside activations

◆ Switched to FSDP — memory per rank dropped from [X]GB (DDP) to [X]GB (FSDP); gradient checkpointing no longer needed, max batch size increased [X]x, samples/sec improved [X]% vs DDP at same GPU count

◆ Benchmarked Ray Train vs torchrun DDP — [X]% throughput difference at 4 GPUs; Ray eliminated manual `dist.init_process_group` setup, added native HPO integration via Ray Tune in under 15 lines

◆ Implemented rank-aware checkpointing — saves on rank 0 only, resumes correctly across all ranks; zero training loss on spot interruption

◆ Logged all stages to MLflow: loss, samples/sec, GPU memory per rank; experiment comparison dashboard shows DDP vs FSDP tradeoff visually

Important: Every [X] must be replaced with a real number from your run. Scaling efficiency is the number interviewers ask about most — know it and be able to explain why it's not 100%.

This is Project 3 of 3. Complete this and all A-tier distributed training skills are covered. Three projects, full resume.